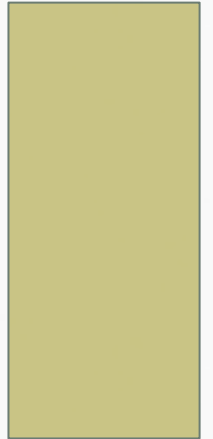


DATA STRUCTURES

LECTURE 6



STACKS

- A stack is a restricted linear list.
- All additions and deletions are made at one end, the top.
- stacks are known as last in, first out (LIFO) data structures.

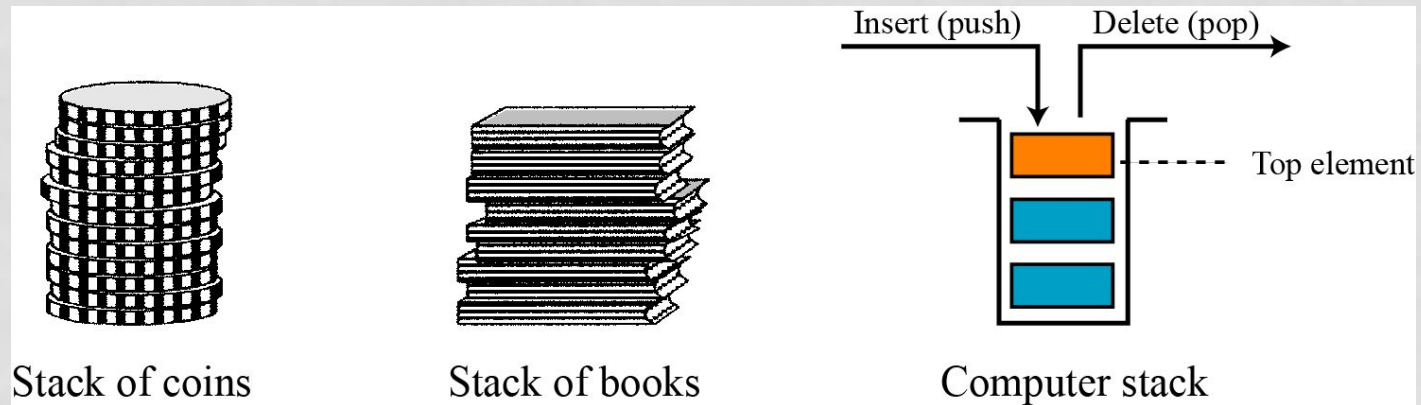


Figure 12.2 Three representations of stacks

APPLICATIONS

- Undo Operations
- **Infix to postfix expressions**
- **Evaluation of postfix expression**
- **Brackets balancing in an equation**
- Activation Record

C++ RUN-TIME STACK

- The C++ run-time system keeps track of the chain of active functions with a stack
- When a function is called, the run-time system pushes on the stack a frame containing
 - Local variables and return value
 - Program counter, keeping track of the statement being executed
- When a function returns, its frame is popped from the stack and control is passed to the method on top of the stack

```
main() {  
    int i = 5;  
    foo(i);  
}
```

```
foo(int j) {  
    int k;  
    k = j+1;  
    bar(k);  
}
```

```
bar(int m) {  
    ...  
}
```

bar
PC = 1
m = 6

foo
PC = 3
j = 5
k = 6

main
PC = 2
i = 5

Operations on stacks

There are four basic operations, *stack*, *push*, *pop* and *empty*, that we define in this chapter.

The *stack* operation

The *stack* operation creates an empty stack. The following shows the format.

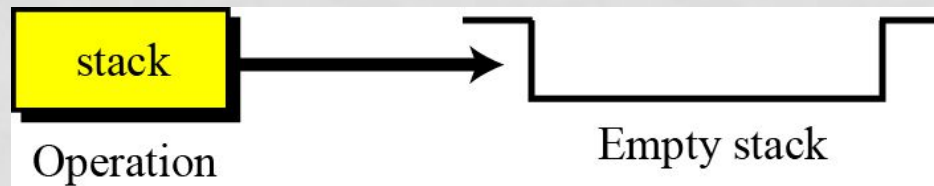


Figure Stack operation

The *push* operation

The *push* operation inserts an item at the top of the stack. The following shows the format.

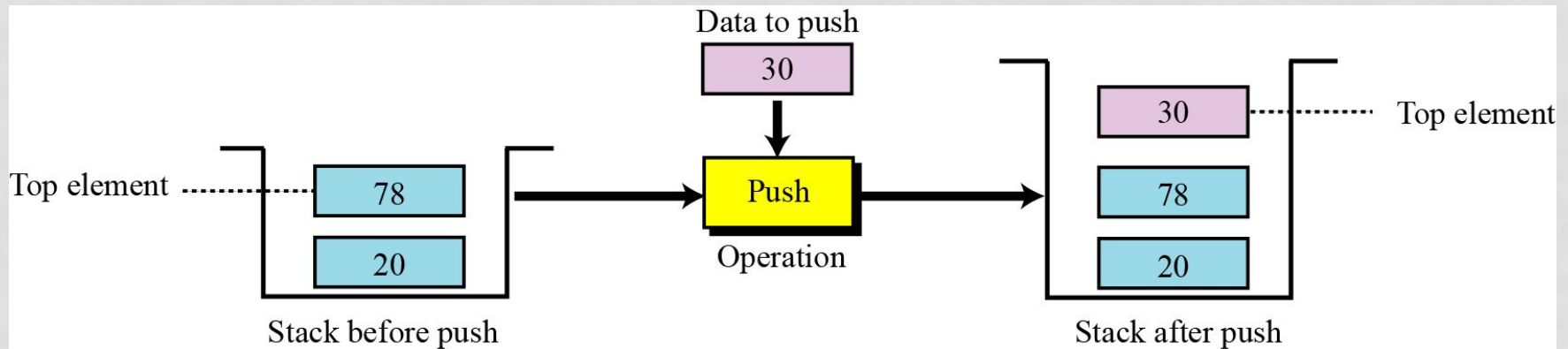


Figure 12.4 Push operation

The *pop* operation

The *pop* operation deletes the item at the top of the stack. The following shows the format.

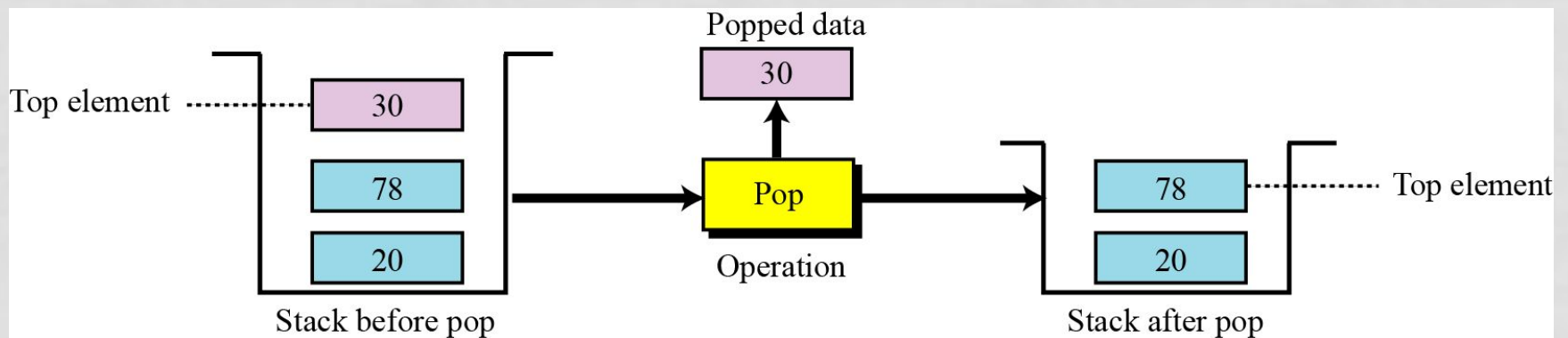


Figure 12.5 Pop operation

The *empty* operation

The *empty* operation checks the status of the stack. The following shows the format.

This operation returns true if the stack is empty and false if the stack is not empty.

STACK IMPLEMENTATION

- Dynamic Implementation
 - Nodes based Implementation
- Static Implementation
 - Array Based Implementation

DYNAMIC IMPLEMENTATION

```
class Stack
{
    Stack (); //Constructor
    bool isEmpty(); //returns true if stack is empty
    void push(int newItem); //pushes newItem at top
    void pop(); //removes top item from stack
    int getTop();
    Stack(const Stack & s);
    ~Stack();
    private:
    node* top;};
```

COPY CONSTRUCTOR

1. Initialize the to be copied stack's top in a temporary pointer "**temp1**"
2. Create a new node "**nnode**"
3. Check if temp1 !=NULL
4. If the new node is going to the first node of the new stack then save it's address in the top of new stack. Copy corresponding value from the other stack.
5. Else
nnode->next = new node; nnode=nnode->next;
6. temp1=temp1->next
7. Go to step 3

DYNAMIC IMPLEMENTATION (USING LINKED LIST)

```
class Stack
{
    Stack (); //Constructor
    bool isEmpty(); //returns true if stack is empty
    void push(int newItem); //pushes newItem at top
    void pop(); //removes top item from stack
    int getTop();
    Stack(const Stack & s);
    ~Stack();
    private:
    Linkedlist<T> l;
};
```

We define a stack as an ADT as shown below:

Stack ADT

Definition	A list of data items that can only be accessed at one end, called the <i>top</i> of the stack.
Operations	stack: Creates an empty stack. push: Inserts an element at the top. pop: Deletes the top element. empty: Checks the status of the stack.

ARRAY BASED IMPLEMENTATION OF STACK

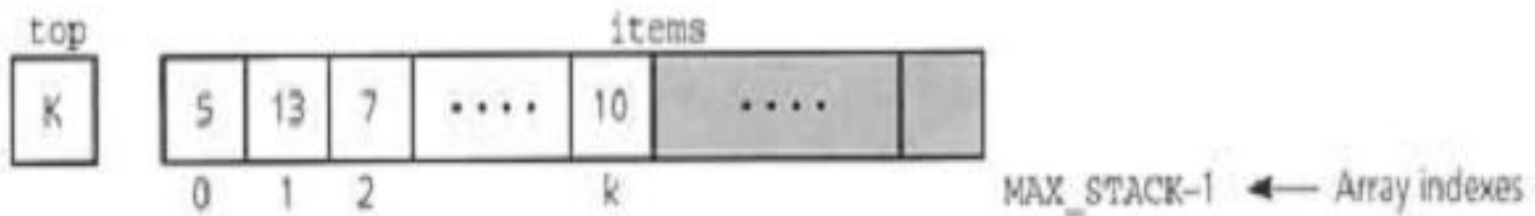


FIGURE 6-5

An array-based implementation

ARRAY BASED IMPLEMENTATION

```
const int MAX_STACK =100;
class StaticStack
{
    StaticStack (); //Constructor
    bool IsEmpty(); //returns true if stack is empty
    void push(int newItem); //pushes newItem at top
    void pop(); //removes top item from stack
    int getTop();
    private:
    int items [MAX_STACK];
    int top;
};
```

IMPLEMENTING STATIC STACK ADT

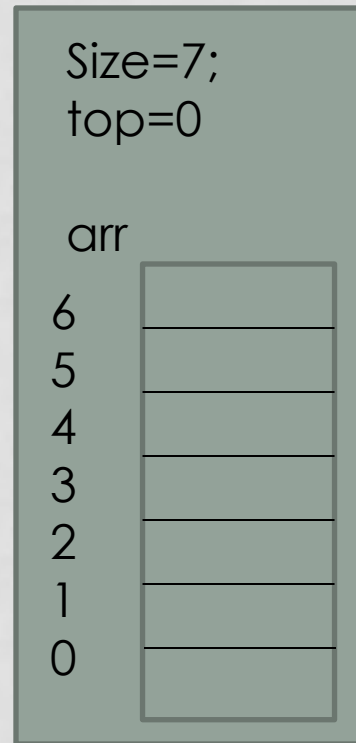
```
void main( )
```

```
{Stack<int> s;  
s.push(6);  
s.push(8);  
s.push(15);  
s.push(18);  
s.push(21);  
s.push(35);  
s.push(45);  
s.push(48);
```

```
s.pop();  
s.pop();  
s.pop();  
s.pop();  
s.pop();  
s.pop();  
s.pop();  
}
```

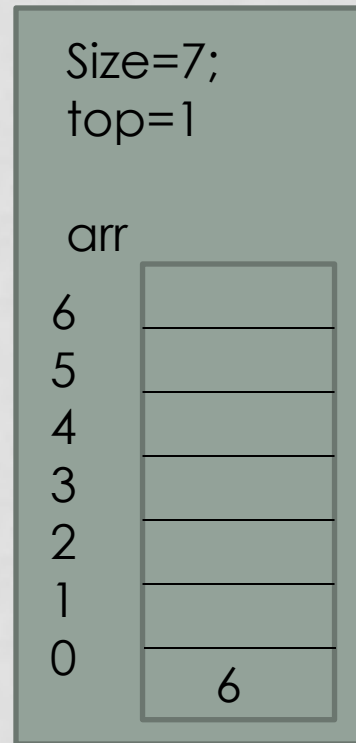

IMPLEMENTING STATIC STACK ADT

```
Stack<int> s;
```



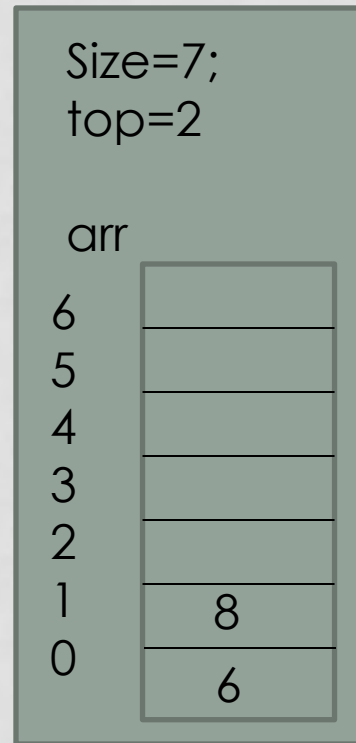
IMPLEMENTING STATIC STACK ADT

s.push(6);



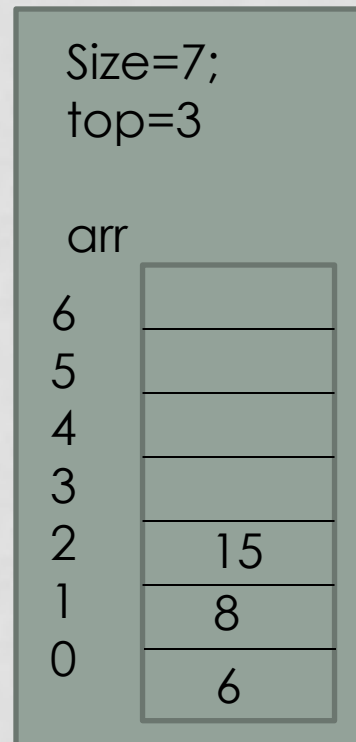
IMPLEMENTING STATIC STACK ADT

s.push(8);



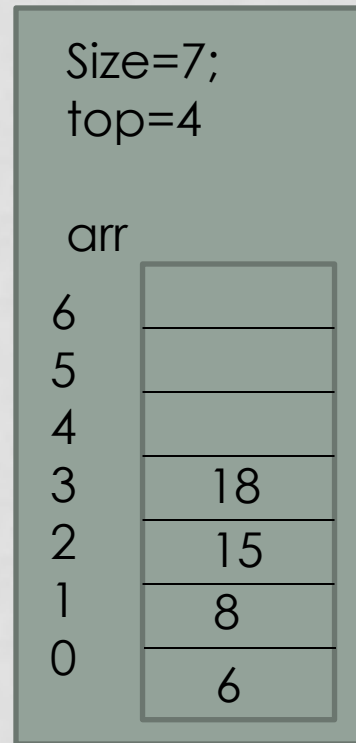
IMPLEMENTING STATIC STACK ADT

s.push(15);



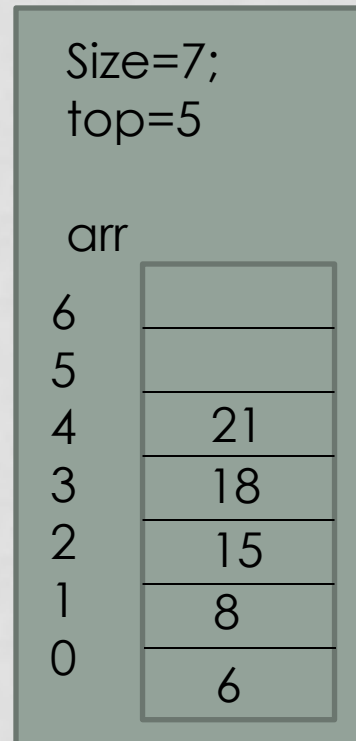
IMPLEMENTING STATIC STACK ADT

s.push(18);



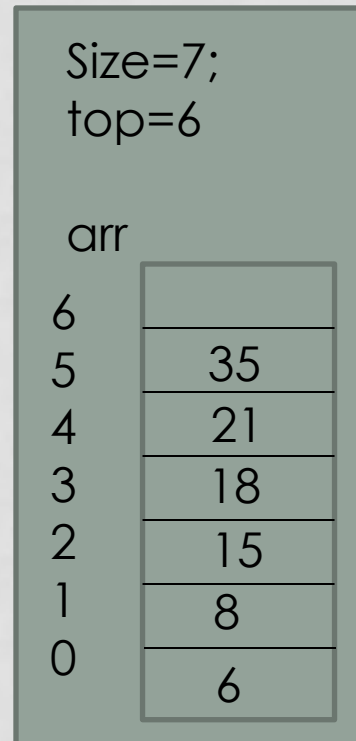
IMPLEMENTING STATIC STACK ADT

s.push(21);



IMPLEMENTING STATIC STACK ADT

s.push(35);



IMPLEMENTING STATIC STACK ADT

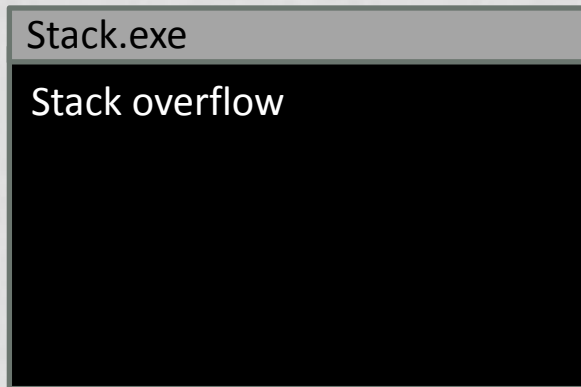
s.push(45);

Size=7; top=7	
arr	
6	45
5	35
4	21
3	18
2	15
1	8
0	6

IMPLEMENTING STATIC STACK ADT

s.push(48);

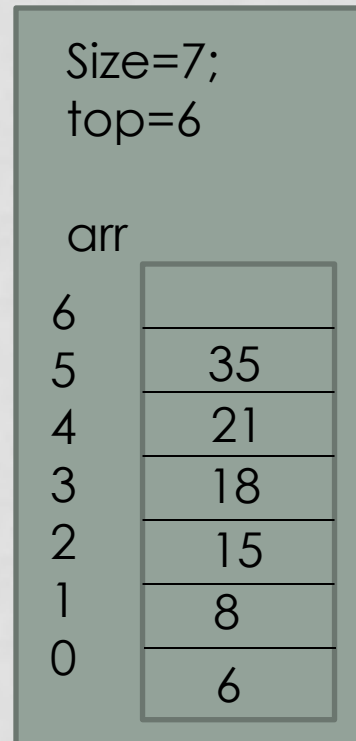
Console



Size=7; top=7	
arr	
6	45
5	35
4	21
3	18
2	15
1	8
0	6

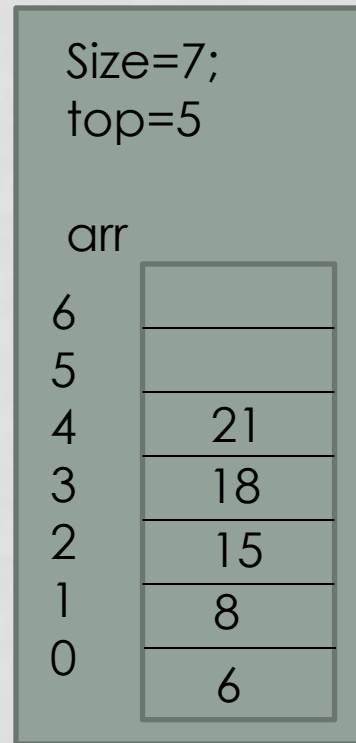
IMPLEMENTING STATIC STACK ADT

s.pop();



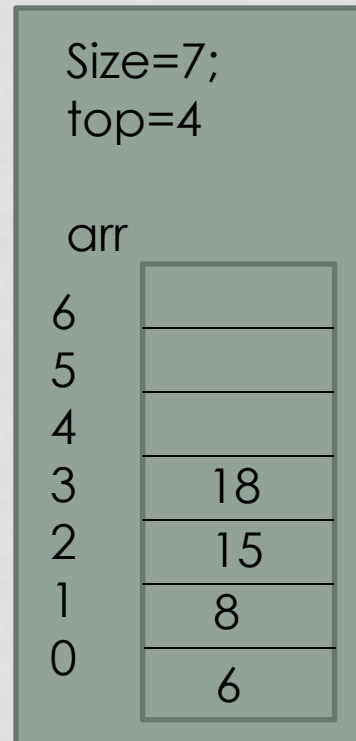
IMPLEMENTING STATIC STACK ADT

s.pop();



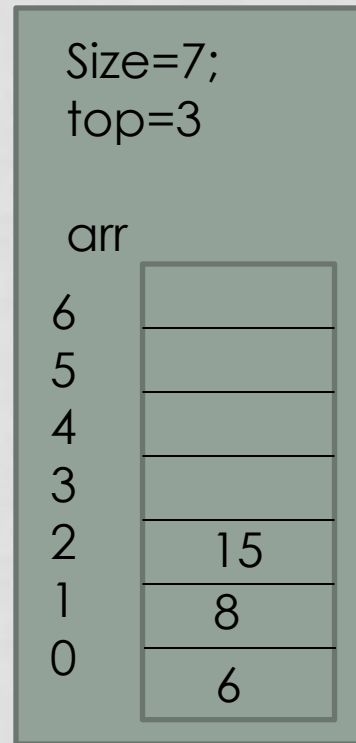
IMPLEMENTING STATIC STACK ADT

s.pop();



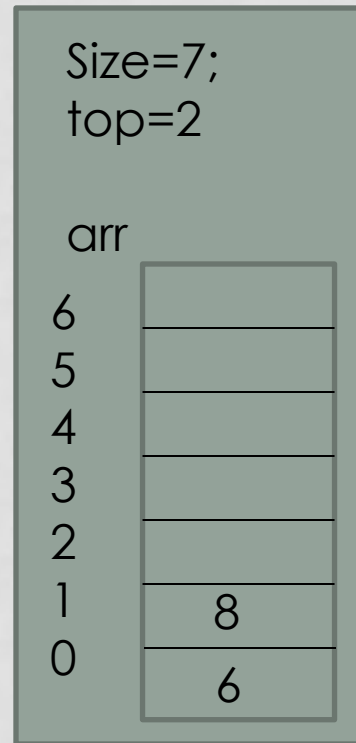
IMPLEMENTING STATIC STACK ADT

s.pop();



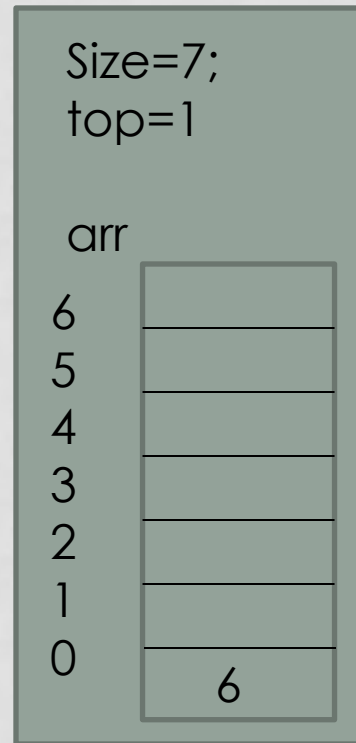
IMPLEMENTING STATIC STACK ADT

s.pop();



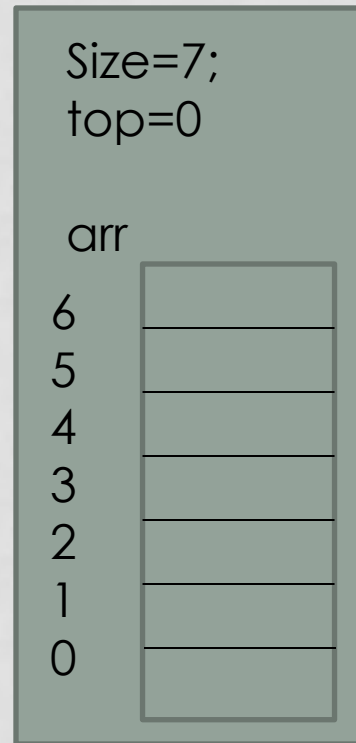
IMPLEMENTING STATIC STACK ADT

s.pop();



IMPLEMENTING STATIC STACK ADT

s.pop();

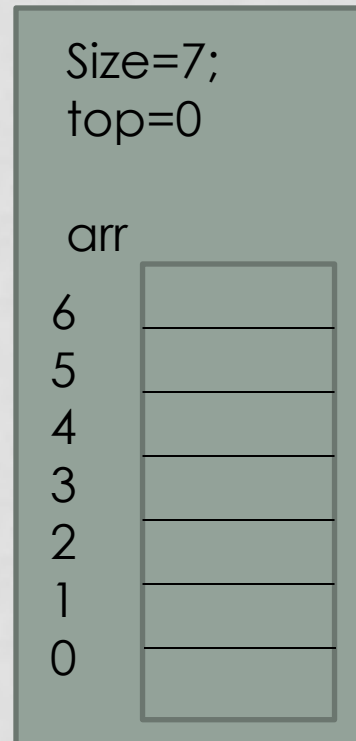


IMPLEMENTING STATIC STACK ADT

`s.pop( );`

Console

```
Stack.exe
Stack overflow
Stack underflow
```



STACK APPLICATIONS

- **Nested Parentheses**

- In mathematical expressions that include sets of nested parentheses; we want to ensure that

1. There are an equal number of right and left parentheses
 $((A+B)$
2. Every right parenthesis is preceded by a matching left parenthesis
 $(A+B))-(C+D$

CHECKING MULTIPLE TYPES OF BRACKETS

String equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"

Let us assume an equation, the algorithm will check

if (equation [i] is an operand)
 continue;

else if (equation[i] is an operator)
 continue;

else if (equation[i] is an opening bracket)
 push(a[i]);

else if (equation[i] is a closing bracket)
 char c= pop();
 if(c matches its corresponding closing bracket)
 continue;
 else
 break;

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[0] is an opening bracket, thus we will push it in stack

A vertical stack diagram consisting of two rectangular boxes. The top box is dark grey and is empty. The bottom box is light grey and contains a single opening curly brace '{' character.

{

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[1] is an operand, thus we will
continue our next iteration

{

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[2] is an operator, thus we will continue our next iteration

{

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[3] is an opening bracket, thus we will push it in stack



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[4] is an operand, thus we will continue to next iteration



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[5] is an operator, thus we will continue to next iteration

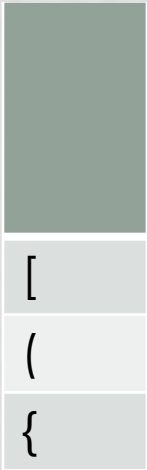


CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[6] is an opening bracket, thus we will push it in stack

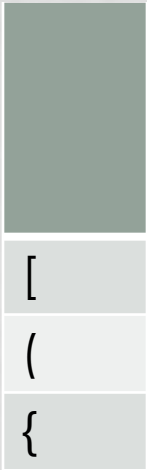


CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[7] is an operand, thus we will
continue to next iteration

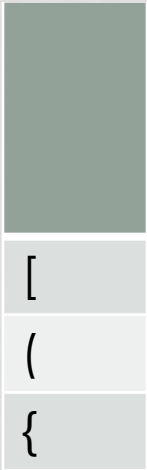


CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[8] is an operator, thus we will continue to next iteration

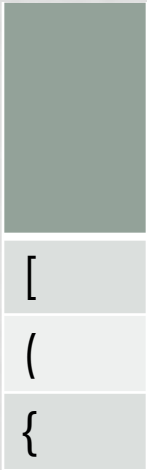


CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[9] is an operand, thus we will continue to next iteration



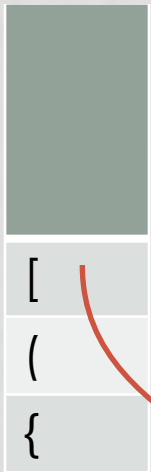
CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[10] is a closing bracket, thus we will pop one element from stack in char ch And match it with Equation[10].

ch contains corresponding closing bracket i.e. '[' and ']' completes the pair, thus algorithm will continue.



Char ch=[

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[11] is a closing bracket, thus we will pop one element from stack in char ch And match it with Equation[11].

ch contains corresponding closing bracket i.e. '(' and ') ' completes the pair, thus algorithm will continue.



Char ch=(

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[12] is an operator, thus we will continue to next iteration



{

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[13] is an operand, thus we will continue to next iteration



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[14] is an operator, thus we will continue to next iteration



}

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[15] is an opening bracket, thus we will push it into stack



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[16] is an operand, thus we will continue to next iteration



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[17] is an operator, thus we will continue to next iteration



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[18] is an operand, thus we will continue to next iteration



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[19] is a closing bracket, thus we will pop one element from stack in char ch And match it with Equation[19].

ch contains corresponding closing bracket i.e. '(' and ') ' completes the pair, thus algorithm will continue.



Char ch=(

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[19] is a closing bracket, thus we will pop one element from stack in char ch And match it with Equation[19].

ch contains corresponding closing bracket i.e. '(' and ') ' completes the pair, thus algorithm will continue.



Char ch=(

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[20] is a closing bracket, thus we will pop one element from stack in char ch And match it with Equation[20].

ch contains corresponding closing bracket i.e. '{' and '}' completes the pair, thus algorithm will continue.



Char ch={

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[21] is an operator, thus we will
continue next iteration

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[22] is an opening bracket, thus
we will push it in stack

A vertical stack diagram. It consists of a tall, empty grey rectangle above a single box containing the opening bracket '('.

(

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[23] is an operand, thus we will continue to next iteration

(

A vertical stack is represented by a grey rectangle. At the bottom of the rectangle, there is a white square containing the opening parenthesis character '('.

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[24] is an operator, thus we will continue to next iteration

(

A vertical stack is represented by two rectangular boxes. The top box is dark grey and is empty. The bottom box is light grey and contains the opening parenthesis character '('.

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[25] is an opening bracket, thus
we will push it in stack



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[26] is an operand, thus we will
continue next iteration



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[27] is an operand, thus we will continue next iteration



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[28] is an operator, thus we will
continue next iteration

[

(

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"

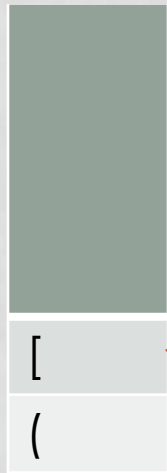


Equation[29] is an operand, thus we will
continue next iteration



CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[30] is a closing bracket, thus we will pop one element from stack in char ch And match it with Equation[30].

ch contains corresponding closing bracket i.e. '[' and ']' completes the pair, thus algorithm will continue.

Char ch=[

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Equation[31] is a closing bracket, thus we will pop one element from stack in char ch And match it with Equation[31].

ch contains corresponding closing bracket i.e. '(' and ') ' completes the pair, thus algorithm will continue.



Char ch=(

CHECKING MULTIPLE TYPES OF BRACKETS

String Equation="{x+(y-[a+b])*c-(d+e)}/(h-[j-k])"



Loop will terminate when string ends. **After loop termination your stack must be empty**

CHECKING MULTIPLE TYPES OF BRACKETS

- Following conditions if achieved will indicate imbalanced brackets in equation
 - If string ends but some opening brackets are still contained by stack. This indicates that no closing corresponding bracket was found
 - If closing bracket is encountered, however when stack is popped for corresponding opening bracket, it doesn't match the closing bracket type.

ALGORITHM

```
valid=true;
s=an empty stack;
while (we have not read the entire string)
{
    read the next symbol (symb) of the string;
    if (symb=='(' || symb=='[' || symb=='{')
        push symb on s
```

ALGORITHM CONTD.

```
if (symb=='(' || symb=='[' || symb=='{')  
    if (s is empty)  
        valid=false;  
    else {  
        i=top item popped from s;  
        if (i is not the matching opener of symb);  
            valid=false;  
        } //end else  
    } //end while
```


ALGORITHM CONTD.

```
if (! (s is empty))  
    valid=false;
```

```
if (valid)  
    cout<<"the string is valid";  
else  
    cout<<"the string is invalid";
```

STACK APPLICATIONS

- Infix Expressions

- An expression in which every binary operation appears between its operands
- Example: (i) $a+b$; here “+” is a binary operation and a and b are its operands

(ii) $(a+b)*c$

- Prefix Expressions

- An expression in which operator comes before its operands
- Example: (i) $a+b \rightarrow +ab$

(ii) $(a+b)*c \rightarrow *+abc$

(iii) $a+(b*c) \rightarrow +a*bc$

STACK APPLICATIONS

- Postfix Expression

- An expression in which operator comes after its operands
- Example: (i) $a+b \rightarrow ab+$
(ii) $(a+b)*c \rightarrow ab+c *$
(iii) $a+(b*c) \rightarrow abc*+$

WHY POSTFIX EXPRESSIONS ARE NEEDED?

- Old compilers: Infix  machine language
- This gets messy because of parentheses.
- Newer compilers: Infix  postfix  machine lang.

INFIX VS POSTFIX

- The order in which operators are applied is always unambiguous – **No need for brackets**
- $a+b*c$
 - $a+(b*c)$ OR $(a+b)*c$
 - Without parenthesis or precedence rules it is not clear
- $abc*+$
- $ab+c*$

INFIX TO POSTFIX CONVERSION

Scan the expression character by character

- **Operands** : send to output string
- **Closed Parenthesis** : Pop stack until open parenthesis and send each element to output string
- **Operator** : Pop stack until we see A symbol of low precedence, then push operator in stack
- **Open Parenthesis** : Push them into the stack

When Expression ends pop all elements from stack to output string

INFIX TO POSTFIX CONVERSION

- IF next symbol is an operand, it is placed into the output stream

Input: $a + b * c - d$

Output: a

opStack: empty



- If the next symbol is an operator,
 - compare the precedence of this operator to the one on top of a stack called opStack, short for *operator stack*

Input: $a + b * c - d$

Output: a

opStack: $+$



INFIX TO POSTFIX CONVERSION

Input: a + b * c - d

Output: a b

opStack: +



Input: a + b * c - d

Output: a b

opStack: + *



Input: a + b * c - d

Output: a b c

opStack: + *



Input: a + b * c - d

Output: a b c

opStack: + *



CONVERTING AN EXPRESSION FROM INFIX TO POSTFIX

```
opstk=the empty stack;
while (not end of input) {
    symb=next input character;
    if (symb is an operand)
        add symb to the postfix string
    else {
        while ( (opstk is not empty) && (operator at top of the stack has precedence
greater than or equal symb)) {
            topsymb=top item popped from opstk;
            add topsymb to the postfix string;
        }
        push symb on opstk;
    } //end else
} //end while
//output any remaining operator
while ( (opstk is not empty) {
    topsymb= top item popped from opstk;
    add topsymb to the postfix string;
} //end while
```

EXAMPLE

Convert to Postfix: $A+B*C$

Symb	Postfix string	opstk
A	A	
+	A	+
B	AB	+
*	AB	+*
C	ABC	+*
	ABC*	+
	ABC*+	

EXAMPLE

Convert to Postfix: $A * B + C * D$

Symb	Postfix string	opstk
A	A	
*	A	*
B	AB	*
+	AB*	+
C	AB*C	+
*	AB*C	++
D	AB*CD	++
	AB*CD*	+
	AB*CD*+	

CLASS WORK

- Convert following infix expressions to postfix expression
 - $A*(B+C)$
 - $(A+B/C*(D+E))-F$

EVALUATING A POSTFIX EXPRESSION

- Each operator in a postfix expression refers to the previous two operands in the expression.
- Each time we reach an operand we push it onto a stack.
- When we reach an operator, its operands will be the top two elements on the stack.
- We then pop these two elements, perform the indicated operation on them, and push the result on the stack.
- In the end, the result of the expression is left as the only element in the stack, which is popped and returned as the result of evaluation

ALGORITHM FOR EVALUATING A POSTFIX EXPRESSION

```
opndstk=the empty stack;  
//scan the input string one element a time into symb  
while (not end of input) {  
  symb=next input character;  
  if (symb is an operand)  
    push symb on opndstk;  
  else {  
    //symb is an operator  
    opnd2=top item popped from opndstk;  
    opnd1= top item popped from opndstk;  
    value=result of applying symb to opnd1 and opnd2;  
    push value on opndstk;  
  } //end else  
} //end while  
return (top item popped from opndstk);
```

EXAMPLE

6 2 3 + - 3 8 2 / + *

Sym b	opnd1	opnd2	value	opndstk
6				6
2				6,2
3				6,2,3
+	2	3	5	6,5
-	6	5	1	1
3				1,3
8				1,3,8
2				1,3,8,2
/	8	2	4	1,3,4
+	3	4	7	1,7
*	1	7	7	7

READINGS

- Chapter 6 , Data Abstraction and Problem Solving using C++, Walls & Mirrors 3rd ed.

HELP

- <https://www.youtube.com/watch?v=1SWr7q121gc>
- <https://www.youtube.com/watch?v=jos1Flt21is>
- [https://www.youtube.com/watch?v=OVFwgYrMSh
w](https://www.youtube.com/watch?v=OVFwgYrMShw)